Final Task for Data Analysis

Davide Orecchioni

AY 2024-2025

Abstract

The first aim of this report is to develop and discuss classification methods to identify whether or not a specimen of the species *Caenorhabditis elegans* belongs to a reference (*wild-type*) strain or to any of four *mutant* strains, based on the `EigenWorms` dataset. The dataset is comprised of time series data regarding the first 6 principal components of a PCA analysis derived from the motion of wild-type worms on agar plates. These components are called *eigenworms*. We analyze four classical methods, first on the raw data and then on lower dimensional data obtained from appropriate feature engineering. We also build a convolutional neural network model to try and capture time-related structure in the series. The second aim of the report is to then give a quantitative measure on the importance each eigenworm has for the success of the methods we introduced.

1 Introduction

1.1 The problem

Caenorhabditis elegans is a species of roundworm historically used as a model organism for research on neural development and behavioral genetics. In [1], Brown *et al.* consider the problem of characterizing behavioral phenotype of wild and mutant strains of *C. Elegans* through the analysis of the characteristics of their motion. Of the many features included in this analysis (such as velocity, length, width of the worm and such) we will focus on the so-called eigenworm projections. These projections are calculated by taking a few wild-type specimens, filming their motion on agar plates and extract motion information from each frame using the method described in detail in [2]. This method consists of parametrizing the worm into segments and calculating angles between these segments in a way that is independent from the choice of coordinates; these angles characterize the worm's shape through time as it moves. A Principal Component Analysis is then performed on the covariance matrix of the angles. The `EigenWorms` dataset contains the time series obtained by projecting each worm's shape at each frame on the first six principal vectors derived from the PCA. These vectors are called *eigenworms*, and they can be seen as base shapes which make up a worm's motion.

1.2 The eigenworms

The six eigenworm considered in the `EigenWorms` dataset make up 97% of worm shape variance, with components 1 through 4 making up 95% of variance. To these first four we can give an intuitive physical interpretation, as more thorougly described in [2].

- The first and second eigenworms can be seen as sinusoidal waves of motion forming a quadrature pair. Amplitudes along these vectors can be seen as different phases of a sinusoidal wave traveling through the worm's body,.which phisically corresponds to the worm moving forward in a straight line. More specigically, crawling backwards or forward in a straight line corresponds to rotating clockwise or counterclockwise in the principal plain generated by the first two eigenworms.
- The third eigenworm is characterized by a constant curvature in the middle of the body. It distribution has a long tail, with amplitudes chosen from those tails corresponding to a Ω-like shape, which occurs when worms turn with a large angle. More generally, large values of amplitudes along the third eigenworm correlate to the presence of turns in the worm's trajectory.
- The fourth eigenworm captures a significant portion of variance in the shape of the head and tail of the worm. These extremities tend to act more independently from the rest of the body, possibly because they have a separate set of muscles.

The main aim of this report is to develop methods for classifying worms into wild-type or mutant based on the time series of their eigenworm projections. To do this, we will first explore the dataset in order to study how the data is composed and make informed decisions on which methods to implement and how.

2 Exploring the dataset

2.1 Train/Test split and class distribution

The dataset comprises of 259 specimens in total, split in 128 Train and 131 Test cases. Each specimen belongs to one of five classes (which we will also call *strains*), with class 1 being wild-type and classes 2 through 5 being mutant. We immediately notice the class distribution is not uniform, with class 5 being noticeably underrepresented. One possible effect of this is the following: when performing k-fold cross-validation, for example with $k = 5$, the hold-out test set has a size of around 50 instances. It is thus possible for every specimen with class 5 to be i the test set; this means the model has not been trained on any class 5 case, which could negatively affect the performance of the model on the test set. On the other hand, if all specimens are in the training set, the model is not tested on any class 5 case, making training less effective. We expect the models to perform best in cases where each class is reasonably split between train and test. This means that, when performing validation, the results could vary greatly depending on the randomly generated sample.

Class	Train	Test
1	55	55
2	22	22
3	17	18
4	22	23
5	12	13
Total	128	131

Table 1: Class distribution

In pratice, we found that when during validation accuracy scores do significantly vary between instances, which is not optimal when comparing the performances of different models. To circumvent this, we will do the following:

- For each method we introduce we will do an initial test on the original Train/Test split provided, where (as seen in Table 1) classes are split in half between train and test. We will use it as a baseline to compare performances of the models, using accuracy as the main metric.
- After all methods have been introduced we will perform k-fold cross-validation on each of them multiple times, to check how accuracy and other metrics fluctuate and then calculate their average to assess the overall quality of the model. After some testing we found that the best choices were $k = 4$ for cross validation using the original data and $k = 5$ when applying data reduction (discussed in Chapter 4).

2.2 Series Inspection and Anomaly Detection

We now turn to some basic visual inspection of the data. To do this, we selected a worm from each class from the training set and plotted the time series of each of the 6 components. Because each series has $N = 17984$ values, for visual clarity we only plotted the beginning of each series.

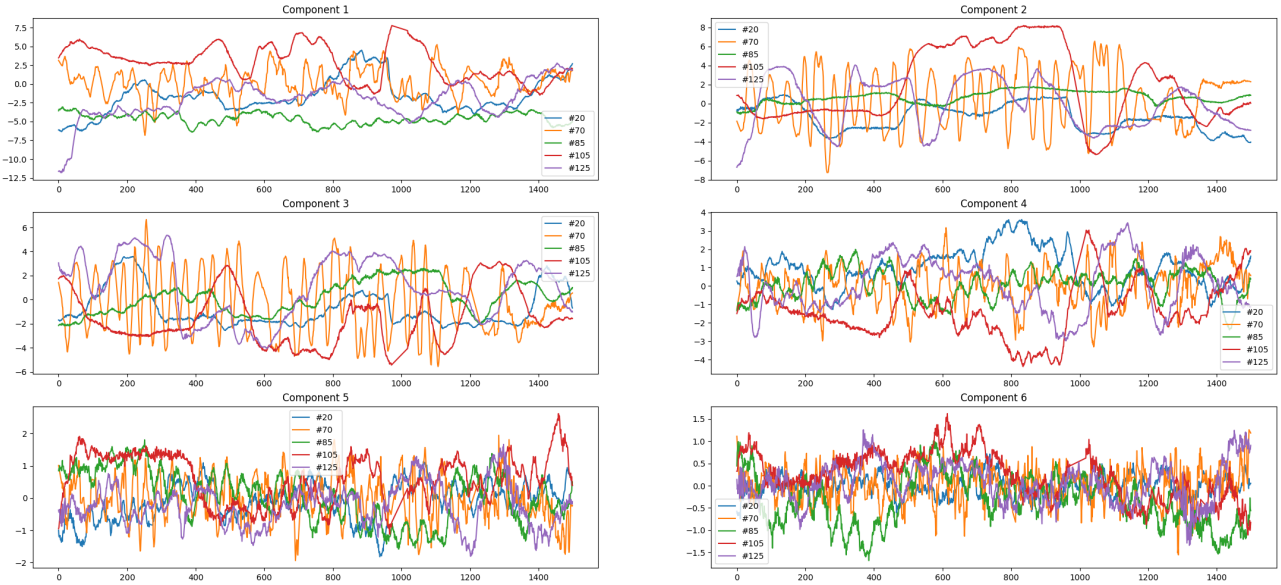


Figure 1: Time series inspection for each class

Some immediate observations from Figure 1 are that values remain relatively close to zero, with noticeably different behavior between classes. For example, the worm belonging to class 3 (plotted in orange) exhibits a very pronounced oscillatory behavior for components $1, 2$ and 3 compared to other classes. We can also notice how the time series of earlier components seem to have behave more regularly, becoming increasingly erratic for latter components (giving the impression of a thicker line on the plot).

Let us now focus on the red line, belonging to the worm from class 4. We can see that, for values preceding 1000 on the x axis, a clear linear trend occurs, especially visible and conspicuous in plots for components 3, 5 and 6. Such phenomenons can be often spotted when inspecting the dataset, and originate from the way the datasets were generated. As described in [1] in the Motion Features section, when computing motion states missing lengths are interpolated linearly from available frames. This procedure is relatively innocuous for a small number of missing frames, but may significantly distort the data when applied for longer times. This is because, as we have seen, values mostly remain close to zero, whereas a linear trend will continue growing away from zero if left unchecked for too long.

To screen for such occurrences we have taken each specimen and computed the mean, standard variation and maximum absolute value reached by its associated time series. This is because we expect all of these values to be clearly impacted by the aforementioned unchecked linear trend if present. Indeed, we have found the following:

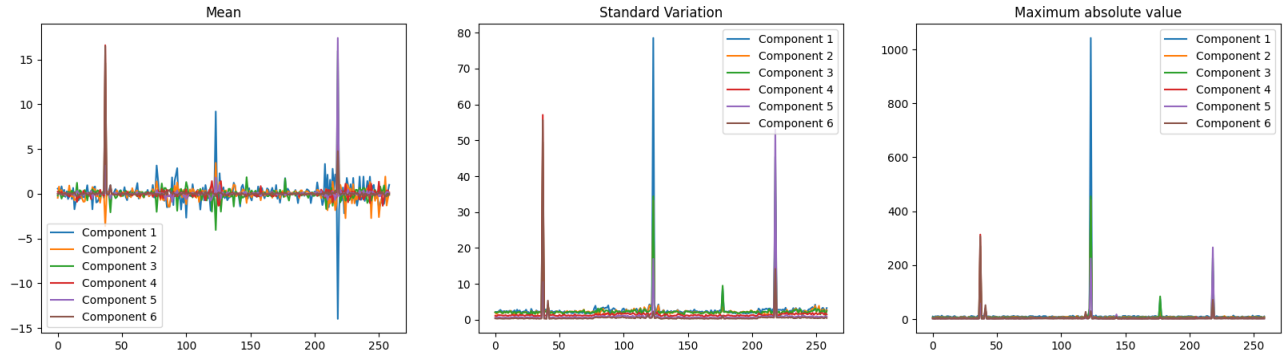
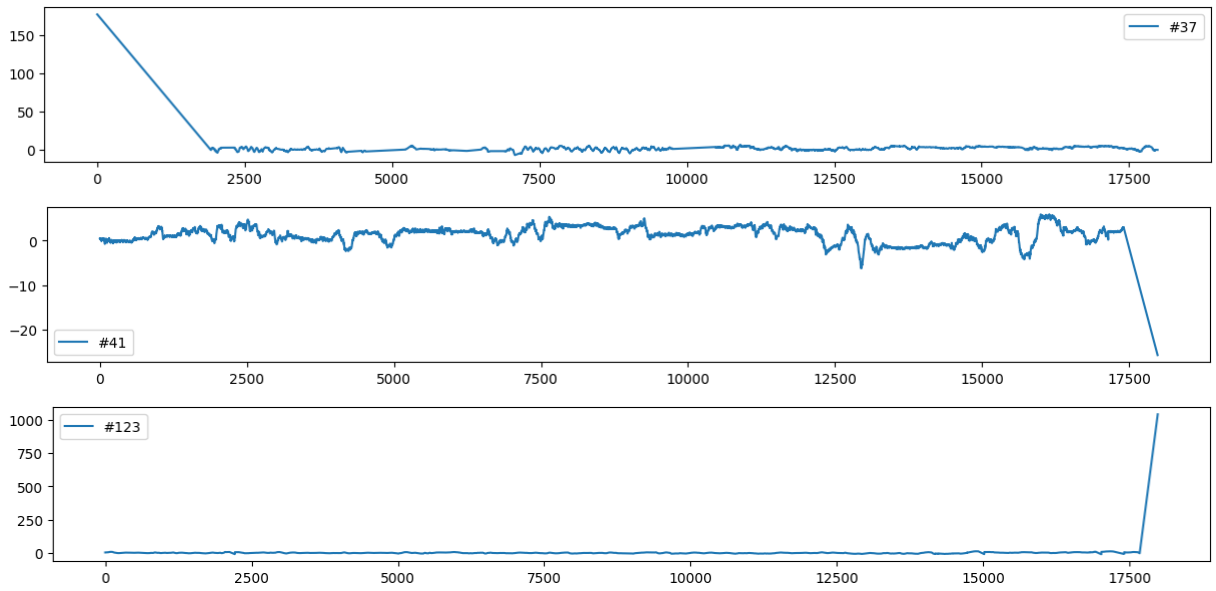


Figure 2: Mean, Standard Variation an Maximum absolute value for the time series of each component

In all three graphs we can notice evident spikes, the biggest of which correspond to the following specimens plotted in Figure 3.



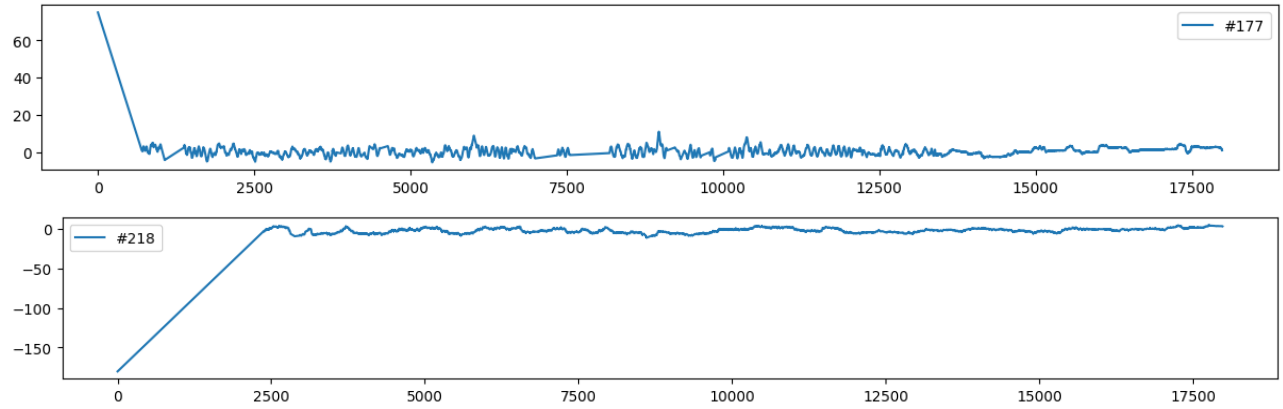


Figure 3: Plot of the first component of worms 37, 41 (Class 1) and 123 (Class 5) from the training dataset and worms 177 (Class 1) and 218 (Class 3) from the test dataset. Plots of other components exhibit similar behavior at the extremes.

In each of these five cases we have an unnnatural linear trend at the beginning or end of the series, which seems to connect an initial (or final) abnormal value to an otherwise well behaved series. Possible explanations for this phenomenon include artificial stretching of shorter time series by to get to the desired length, or simple human error in the measuring and processing phases. It is worth noting that, while these are the most egregious cases, other specimens may exhibit less pronounced but still misleading linear trends.

In the following analysis, the aforementioned 5 cases have been marked as outliers and excluded from the data. This had a positive (albeit small) effect on the accuracy of the raw data methods, which we will now introduce in Chapter 3. The improvement was more noticeable for the feature reduction approach of Chapter 4, which is consistent with the fact it takes into account mean and standard variation.

3 Classical methods

3.1 The methods

In this chapter we will consider four classical methods and compare them on the original train/test split provided by the dataset using accuracy as the main metric, as described in Chapter 2. The methods will be applied to the raw time series data, flattening each series to a row vector where columns (features) represent the value of one of the series at a certain time. This is done for every component and the six row vectors thus obtained are then concatenated to obtain our data matrix, with the last column being `strain` (a.k.a. class, from 1 to 5). The resulting data is of very high dimension, and can thus be relatively unwieldy from a computational point of view. Furthermore, time-dependent imformation may be lost; part of our analysis will hinge on evaluating whether or not how behavior changes through time is relevant to our classification attempts.

The methods we will consider are the following:

- `Logistic Regression` and `K-Nearest Neighbors` were chosen for being among the simplest classification methods. We will use their performance as a reference against more complex methods.
- `Random Forest` was chosen as it is often achieves good results in cases where the dimension of the data is much greater than the number of samples (259 vs $6 \times 17984 \sim 10^5$) with a contained computational cost.
- `Support Vector Machines` were also chosen for being well suited to high dimensional feature spaces.

3.2 Our approach

Theoretically all methods above can be applied to multiclass problems, while ours is binary (wild-type or mutant). Because we do have access to class information, we can approach the task in two distinct ways.

- **1-5 Approach:** we train the model to predict the actual class of each instance, then separate the output into wild-type (1) and mutant (2, 3, 4, 5) and calculate the accuracy.
- **0-1 Approach:** we separate inputs into wild-type and mutant, discarding class information, and train the model to predict whether each worm is mutant or not.

In principle we expect the 0-1 approach to yield better results, as models have more mutant data to train on and the classification task is simpler than identifying a specific strain. However, depending on how the data points are distributed, there may be advantages to the 1-5 approach. For example, it might be easier for SVM to linearly separate singular strains from each other rather than separating all mutant strains from wild-type data points. Similarly, KNN could over or underperform depending on if and how specimens of each class cluster together.

In addition to the two approaches we also consider the accuracy in predicting the actual class, to gain extra insight on how the method is learning the problem. Of course, accuracy scores will be strictly worse than those of the 1-5 approach.

Hyperparameters for KNN were chosen through the `GridSearchCV` method with 4-fold validation (as discussed in Chapter 2, we chose a lower k to reduce fluctuations in the testing error). The best parameters found were `n_neighbors = 4` and `weights = 'distance'`, meaning each neighbor's contribution is weighted based on the inverse of its distance. In the case of Random Forest, where using `GridSearchCV` is significantly more time consuming, we plotted a graph of accuracy values on the default train/test split and used it to select parameters `n_estimators = 250` and `max_depth = 4`. In both cases the optimal parameters were chosen starting from non-scaled data, but they can be empirically shown to be optimal or close to optimal for rescaled data as well.

Lastly, we consider whether or not we need to rescale data. From our series inspection we expect most values to be of similar scale, meaning rescaling may not be as necessary. Of the methods considered, LR prefers scaled data to ensure convergence, and SVM is also known to perform better after rescaling. In practice, we have found that standardization (subtracting the mean and dividing by standard variation) does significantly improve accuracy for both methods when compared to the original data, and also performs better than min-max scaling. In the following sections, we will test the accuracy of all methods with and without standardization.

3.3 Results

Let us now review the results of our four methods. All methods achieve a train accuracy of about 97%, with SVM performing slightly worse but only in the non-standardized case. We now focus on test accuracy.

Method	Without rescaling	With rescaling
Logistic Regression:		
Class accuracy	27.13%	37.21%
1-5 accuracy	58.14%	62.02%
0-1 accuracy	58.14%	62.02%
KNN:		
Class accuracy	51.94%	51.16%
1-5 accuracy	54.26%	51.94%
0-1 accuracy	58.91%	51.94%
Random Forest:		
Class accuracy	51.94%	51.16%
1-5 accuracy	51.94%	51.16%
0-1 accuracy	75.96%	70.54%
SVM:		
Class accuracy	50.39%	65.12%
1-5 accuracy	50.39%	76.74%
0-1 accuracy	72.87%	82.17%

Table 2: Test accuracy of all methods on the original train/test split

As expected, Logistic Regression performs better after standardization; it is by far the worst method for multi-class performance, but is overall better than KNN for binary classification with both the 1-5 and 0-1 approaches. KNN has generally bad performance, and is noticeably negatively affected by data standardization. A possible explanation could be distance-based weighing interacting negatively with high dimensional data.

Random Forest overall achieves solid results, being marginally worse for standardized data. Regarding this, it is worth noting that, unlike all other methods, RF depends on a random seed, which was fixed in the testing phase for consistent results. We will later give a more robust measure of its perfomance by averaging multiple results when doing cross-validation. In general, while the 1-5 approach is not very successful, RF is significantly better than previous methods for the 0-1 approach, achieving consistently better results than LR even in the less favorable standardized scenario.

Finally, SVM is by far the best performing method, with an unparalleled 82% accuracy after standardization (which, as we expected, significantly improves overall performance). Compared to the other methods it achieves better multiclass results, and is the first method for which 1-5 accuracy is not far behind 0-1 accuracy.

Overall SVM proved to be the best method when applied on the raw time series data, coherently with the fact it is designed for dealing with high dimensional data. On the other hand, RF is not far behind at least for the 0-1 approach, with a respectable 75% accuracy (depending on the seed). As a reminder, Random Forest works by uniformly sampling an appropriately small amount of features and building decision trees based on those features. Thus, Random Forest's relative success suggests that we do not need a huge number of features (in our case, time based measurements) in order to successfully classify mutant specimens. This insight is what inspired a different approach to the data, which we will now explore in Chapter 4.

4 Dataset Reduction

4.1 Feature engineering

As discussed in Chapter 3, the fact Random Forest manages to attain a good test accuracy while using a small number of observations from the series suggests we can get away with not using all of the data. We thus turn to a feature engineering approach, where we try do extract information from each of the series using a small number of features for each. This process drastically reduces the dimension of the data, which also helps with computational cost. Of course, we want to choose features that help distinguish different classes from each other.

To identify such features we can once again start from inspecting the series. A first factor which may differentiate classes from each other is relative position in the plots, as we can see in Figure 1. For example, in the plot for component 1, class 3 (in green) seems to consistently assume lower values compared to other classes, meaning its plot stays below them; the exact opposite holds for class 4 (in red). To capture positional relationships of this kind we chose features `Mean`, `Min` and `Max`.

Going back to our series inspection in Chapter 2, we noticed how class 2 (in orange) has a more pronounced oscillatory behavior compared to other classes for components 1, 2 and 3. A simple way to capture this behavior is to use the `Standard Variation` feature. However, standard variation alone doesn't tell us if a series oscillates rapidly around its mean, like for example class 2 in the graph for component 2, or if it has a more stable behavior with wider peaks and valleys, like class 4 in the same plot. To give a measure of this phenomenon we considered the vector of differences and counted the amount of times its absolute value surpasses a certain threshold, which relates to the steepness of the line in the plot. Such a measure would be high for class 2, but low for class 4, distinguishing them in a way the previous features may not be able to. We called this feature `ZZ` (which stands for zigzag). To choose the steepness threshold we looked at plots for difference vectors (some examples can be found in the Colab file). Because most of the oscillation of the data is contained in the $[-0.2, 0.2]$ range, we chose the threshold to be 0.25; this value proved to be a good compromise during testing.

We have thus selected five representative features, that extract data from each of the time series. Because we have one time series for each of the six components, the resulting dataset has a total of 30 features, making it much more manageable than the original one. Furthermore, the reduction process only needs to be done once for the data to be available for multiple methods. Once again, however, all time-dependent information is lost.

4.2 Results

Compared to the original data we expect rescaling to be more relevant as we cannot expect all features to have similar scales, especially in the cases of `SD` and `ZZ`. In practice we found that all of our methods have better performance after rescaling, both for training and test accuracy. Logistic Regression and SVM are massively improved by standardization, and during experimentation we witnessed how LR can struggle to converge when fed non-scaled data. KNN has a smaller but still noticeable improvement, whereas Random Forest is mostly unaffected by the scaling, with slight differences in accuracy partly explained by its dependence on the random state. Once again KNN hyperparameters were chosen using `GridSearchCV`; the best parameters found were `n_neighbors = 5`, `weights = 'distance'`. As for Random Forest, using the same method we found parameters `n_estimators = 100` and `max_depth = 5`. The reduction in the optimal number of estimators is coherent with the dimensional reduction. The cross validation used for this selection was $k = 5$, which in testing proved to give better results for the reduced data.

Based on our findings above we will focus on accuracy results for scaled data, using the same train/test split as in Chapter 3. For training accuracy KNN mantains a value of 97% while other methods achieve around 92% to 96%, which is still very acceptable. Let us now focus on test accuracy.

Logistic Regression:	
Class accuracy	82.95%
1-5 accuracy	95.35%
0-1 accuracy	89.92%
KNN:	
Class accuracy	81.40%
1-5 accuracy	87.60%
0-1 accuracy	89.92%
Random Forest:	
Class accuracy	80.62%
1-5 accuracy	93.02%
0-1 accuracy	93.02%
SVM:	
Class accuracy	83.72%
1-5 accuracy	93.02%
0-1 accuracy	92.25%

Table 3: Test accuracy on the original train/test split after reduction

As we can see in Table 3, the slight drop in training accuracy is more than compensated for by a big improvement in testing accuracy. Interestingly KNN, which was the only method not to lose any training accuracy, has slightly worse performance than the other methods, but still has overall good performance. Compared to our results in Chapter 3, we can immediately notice how every method becomes much better in assigning the correct class, confirming the features we engineered are in fact useful to discriminate between different classes. A very significant result related to this improvement is that with the exception of KNN the 1-5 approach is actually more successful than the 0-1 approach. The difference is minor for RF and SVM, which perform very similarly, and quite noticeable for Logistic Regression which achieves an admirable 95% accuracy. This confirms how, while 0-1 can be generally expected to perform better, the way data is distributed can make the 1-5 approach more successful, like we discussed in Chapter 3. Of course this assumes we have complete strain information; in cases where we only have mutancy information we have to take 0-1 approach which as Table 3 illustrates is almost equally as effective.

4.3 Validation

While results of our data reduction method look promising, we have for now only tested on the original train/test split. To give a more robust measure of the performance of the data reduction approach we now turn to k-fold cross validation. During testing, we performed some validation on the original data for the methods introduced in Chapter 3, and found that choosing $k = 4$ did somewhat curb the instability in accuracy caused by the unbalanced classes, as discussed in Chapter 2. Despite this, results were still rather erratic, with huge swings of up to more than 20 percentage points between different tries. Because of this instability and of the generally worse performance of the raw data methods compared to data reduction ones, we do not report the numerical results for cross-validation for the methods of Chapter 3. They are however available on the Google Colab file, in the Validation section. We now focus on the results of the data reduction approach.

In the case of data reduction we found $k = 5$ to be the best parameter for cross-validation, possibly because of the reduced dimension making models behave less erratically. We ran each method 10 times, calculating accuracy, precision, recall and F1 score for each of them. Results for all metrics can be found in the Colab file. Lastly, we averaged these results to give a more robust measure of their performance. Reults for each of the 10 instances can again be found in the Colab file; average values for all metrics are displayed in Table 4[1].

[1]It is worth noting that when running the code again, cross validation will yield different results because of the innate randomness of the cross-validation process. Because of this, values in the Colab file might not coincide with those in Table 4.

Method	Av. Accuracy	Av. Precision	Av. Recall	Av. F1 score
LR — Multiclass	84.90	84.90	84.90	84.90
LR — 0-5	93.53	92.94	93.4	92.82
LR — 0-1	90.78	90.02	90.38	89.75
KNN — Multiclass	83.33	83.33	83.33	83.33
KNN — 1-5	93.92	91.23	95.79	93.29
KNN — 0-1	94.31	92.33	95.38	93.7
RF — Multiclass	84.12	84.12	84.12	84.12
RF — 1-5	92.55	91.24	92.8	91.79
RF — 0-1	91.76	92.38	89.96	90.84
SVM — Multiclass	86.27	86.27	86.27	86.27
SVM — 1-5	93.92	93.54	93.46	93.14
SVM — 0-1	93.14	93.95	91.33	92.23

Table 4: 5-fold cross-validation average results for Accuracy, Precision, Recall and F1 Score.

The results of cross-validation seem to widely agreee with values on the original test/train split, with similar accuracy scores for all methods. We notice a significant improvement for KNN, likely because of the increased number of neighbors to train on. Based on these results, we can conclude that data reduction is a winning approach to the data for classification.

Overall the data reduction approach was a remarkable success, improving the performance of all of four classical methods while having a very noticeable advantage in terms of manageability and size of the data. We have also made full use of our knowledge of the true strain labels, massively improving multiclass and 1-5 performance. However, the reduction process completely destroys time-related structure in each of the series, meaning there could still be untapped potential to further improve our classification abilities. To try and capture such additional information we will introduce an ad-hoc Neural Network-based method, which we now describe in Chapter 5.

5 Neural Networks

5.1 Convolutional Neural Networks

Convolutional Neural Networks are a category of widely used methods with many applications, one in particular being image processing[2]. The main idea is using layers of convolutional filters to extract deep information from the data. MaxPool layers are added to make the model more robust to small variations in subsequent values. They are also useful in reducing dimensionality, which is also achieved with a stride factor on the convolutional layer. After convolution the resulting data is flattened into a single vector, which is then fed to a series of dense layer and finally to the output layer. A more thorough description of the structure we used can be found in [3].

In our case, because the end goal is time series classification, the final layer will have a neuron for each class (5 for the 1-5 approach, 2 for the 0-1 approach) with the `softmax` activation function. The predicted label will correspond to the neuron with highest output. In the binary case we can also take a slightly different approach and use single neuron in the final layer with the `sigmoid` activation fuction. The final output can now be interpreted as the probability that a specimen is mutant or not. In the testing phase, we analyzed all of the above possibilities on the original test/train split using the raw time series data. This is because our end goal is using these models to try and capture time-related information that could get lost in the reduction process.

[2]Part of the inspiration for using CNNs came from the Intelligent Systems for Pattern Recognition course, where we discussed CNNs in detail and were also tasked with employing them to build a denoising autoencoder for image processing.

5.2 Results

Training was performed for 25 epochs, as during testing we found methods to improve very little after around 20 epochs. Because we are dealing with Neural Networks, it is usually best to use standardized data. In practice we found that results on non-scaled data were not far from results for the scaled case, but the methods can incur into difficulties with convergence. The following results refer to calculations on standardized data. Of course, results may vary when training the model multiple times due to the probabilistic nature of NN training algorithms[3].

5 neurons, `softmax`:	
Class accuracy	74.42%
1-5 accuracy	80.62%
2 neurons, `softmax`:	
0-1 accuracy	85.27%
1 neuron, `sigmoid`:	
0-1 accuracy	84.50%

Table 5: CNN test accuracy on the original train/test split

Overall, CNN methods achieve a respectable performance compared to classical methods on the raw data, having better success with the 0-1 approach. This suggests that, compared to the methods in Chapter 3, convolutional neural networks do manage to capture more of the structure of the problem, which includes time-dependent information. The performance of CNN methods could be further improved by adding convolutional layers or by increasing the number of neurons in the dense layers. These improvements however come at the cost of increased complexity and training time, which is not optimal, especially in our case where we have better performing, lighter models.

While the models we constructed are relatively light, they still require a considerable amount of time for training; this becomes hard to manage when, for example, we want to train the model multiple times during cross-validation. The time cost is especially hard to justify considering how the data reduction approach of Chapter 4 is both more effective in terms of accuracy and more efficient in terms of computing time. For these reasons, we have chosen to skip cross-validation for neural network based methods. The same decision was made for component importance analysis, which is the next and final part of our report.

6 Component importance

6.1 Leave One Out analysis

In the first five chapters we have focused on the first task of this report, which was developing method for time series-based classification. We now turn to our second and final task, which is to understand which of the components (eigenworms) are more relevant to our methods. To do this we used a Leave One Out approach: we trained each model 6 times, each time subtracting information related to one of the eigenworms from the dataset and calculating accuracy. We then compared our results with the accuracy values achieved by models trained on the full data.

The Leave One Out procedure was applied on both the original data and on the reduced data, using the original train/test split. This is because, as discussed, methods on the original data have huge fluctuactions when tested on random samples. In the case of reduced data, comparing tables 3 and 4 we know results are relatively similar, meaning the results of the Leave One Out analysis will also be relevant for different train/test splits.

The following figures and tables contain the results of our Leave One Out analysis. The graphs are to be read with the following convention: the column $x =$ `None` contains values for methods trained on the complete datase, whereas columns $x = i$ correspond to methods trained on all components except i for $i = 1 \dots 6$.

[3]Again, like for cross-validation, running the code in the Colab file may yield different values than those in Table 5.

To assess the importance of each component we need to compare whether accuracy improves or deteriorates (compared to the value for complete data) when leaving out a specific component. Graphically, if the plot rises for $x = i$, i.e. accuracy improves when leaving out data for eigenworm i, we can deduce this eigenworm to be less relevant to the classification method. Conversely, if the plot goes down i.e. accuracy drops, we expect the eigenworm to be more relevant. Ordering components based on the difference of the accuracies we get a component importance ranking, with the most relevant component corresponding to the minimum of the differences. These differences also act as a quantitative measure of relevance, and can be found in tables 6 and 7.

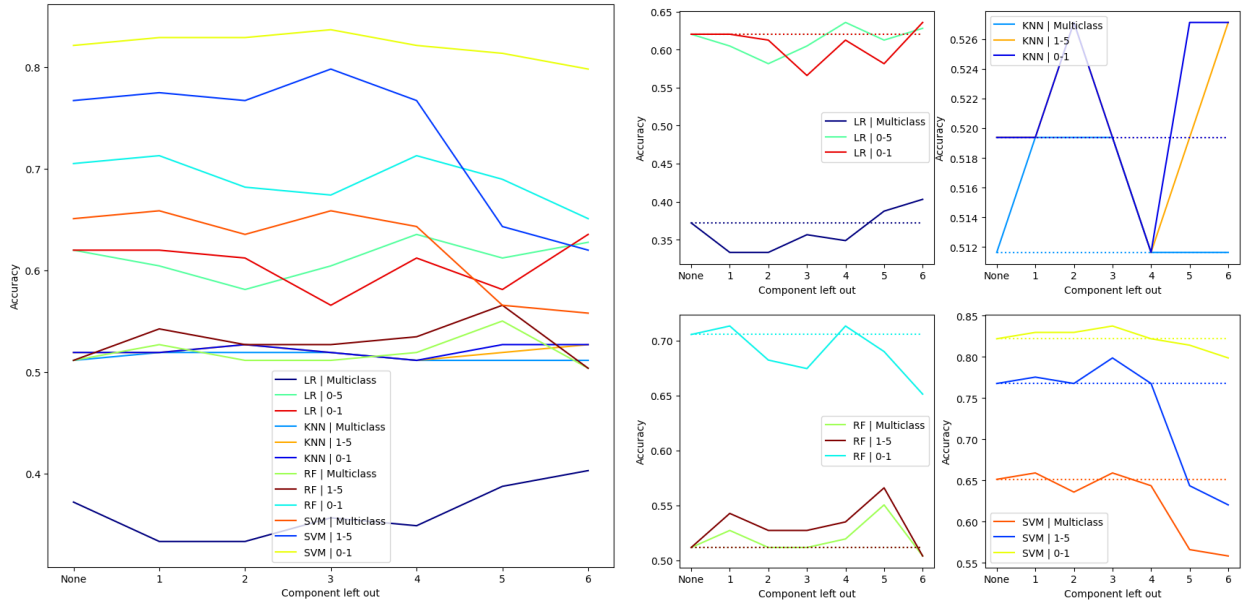


Figure 4: Accuracy results for Leave One Out analysis on the original data. On the left we have all methods, to compare relative performance. On the right we separate them to better focus on the effect of leaving out each component. The dotted line corresponds to accuracy on the full data.

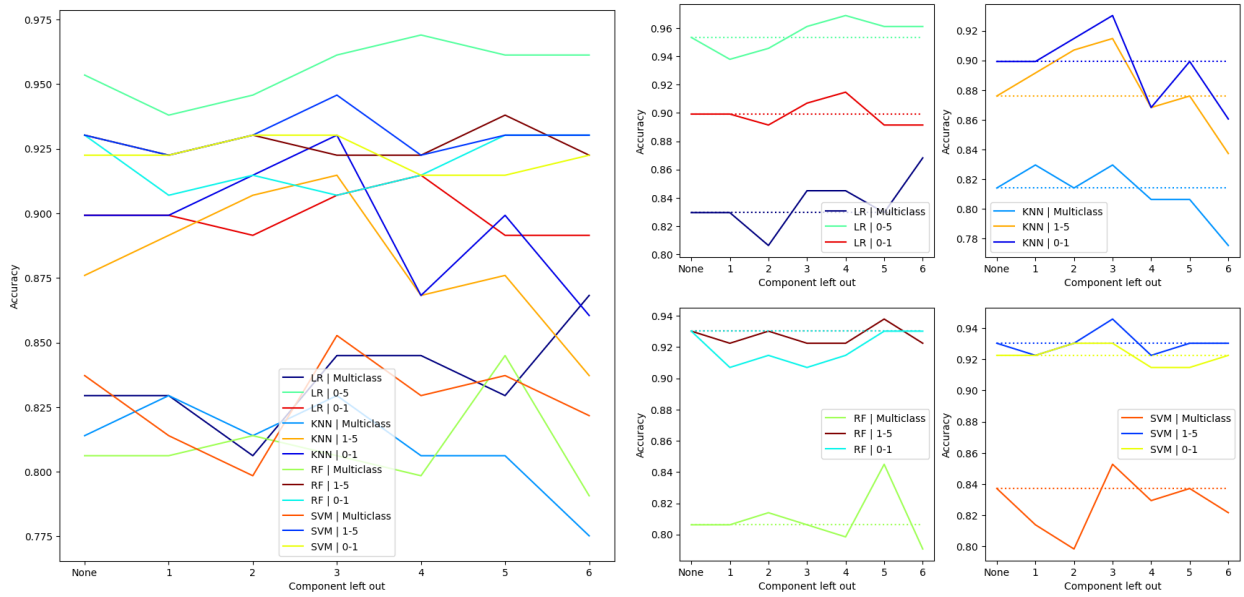


Figure 5: Accuracy results for Leave One Out analysis on reduced data. The layout is the same as Figure 4.

Method	None	1	2	3	4	5	6
LR — Multiclass	37.21	-3.88	-3.88	-1.55	-2.33	1.55	3.1
LR — 0-5	62.02	-1.55	-3.88	-1.55	1.55	-0.78	0.78
LR — 0-1	62.02	0.0	-0.78	-5.43	-0.78	-3.88	1.55
KNN — Multiclass	51.16	0.78	0.78	0.78	0.0	0.0	0.0
KNN — 1-5	51.94	0.0	0.78	0.0	-0.78	0.0	0.78
KNN — 0-1	51.94	0.0	0.78	0.0	-0.78	0.78	0.78
RF — Multiclass	51.16	1.55	0.0	0.0	0.78	3.88	-0.78
RF — 1-5	51.16	3.1	1.55	1.55	2.33	5.43	-0.78
RF — 0-1	70.54	0.78	-2.33	-3.1	0.78	-1.55	-5.43
SVM — Multiclass	65.12	0.78	-1.55	0.78	-0.78	-8.53	-9.3
SVM — 1-5	76.74	0.78	0.0	3.1	0.0	-12.4	-14.73
SVM — 0-1	82.17	0.78	0.78	1.55	0.0	-0.78	-2.33

Table 6: Accuracy differences for the original data. The `None` column contains accuracy for models trained on complete data. Columns 1 through 6 contain the change in accuracy when leaving out the related eigenworms. The lower the value, the higher the eigenworm is likely to be relevant for classification.

Method	None	1	2	3	4	5	6
LR — Multiclass	82.95	0.0	-2.33	1.55	1.55	0.0	3.88
LR — 0-5	95.35	-1.55	-0.78	0.78	1.55	0.78	0.78
LR — 0-1	89.92	0.0	-0.78	0.78	1.55	-0.78	-0.78
KNN — Multiclass	81.4	1.55	0.0	1.55	-0.78	-0.78	-3.88
KNN — 1-5	87.6	1.55	3.1	3.88	-0.78	0.0	-3.88
KNN — 0-1	89.92	0.0	1.55	3.1	-3.1	0.0	-3.88
RF — Multiclass	80.62	0.0	0.78	0.0	-0.78	3.88	-1.55
RF — 1-5	93.02	-0.78	0.0	-0.78	-0.78	0.78	-0.78
RF — 0-1	93.02	-2.33	-1.55	-2.33	-1.55	0.0	0.0
SVM — Multiclass	83.72	-2.33	-3.88	1.55	-0.78	0.0	-1.55
SVM — 1-5	93.02	-0.78	0.0	1.55	-0.78	0.0	0.0
SVM — 0-1	92.25	0.0	0.78	0.78	-0.78	-0.78	0.0

Table 7: Accuracy differences for reduced data. The layout is analogous to Table 6.

6.2 Discussing results

In the case of the original data, looking at the plots we can notice how the four best performing methods (namely, 0-1 Random Forest and all three variations on SVM) all have noticeable accuracy drops when holding out components 5 and 6. The drop is especially significant in multiclass-related methods, as we can see both in the SVM subplot and in Table 6. This suggests that components 5 and 6 encode class-related information, and are the most relevant for the considered methods on the original data. Because this happens for all of our best performing methods, it is safe to say eigenworms 5 and 6 are the most relevant for classification purposes. This is particularly interesting since components 1 through four explain 95% of variance, with 5 and 6 adding just 2% for a total of 97%. While seemingly less relevant in terms of variance, to the point one may choose to drop them in favor of a lighter model, these eigenworms insted contain important information. We also note how in the case of KNN, the most relevant component seems to be number 4, but the drop in accuracy is not particularly significant.

When looking at reduced data, component number 4 again seems more relevant for KNN, this time together with component 6. A few other methods have a drop when leaving out components 1 or 2, which could be explained by these components having a larger. In general, none of the methods exhibit large drops of accuracy for any of the eigenworms like for the original data, and no strong general trend emerges. This could be partly explained by the reduction process erasing most of the structure of the series, including time-based information. On the other hand, it might mean that data reduction adds a level of robustness which prevents huge drops in accuracy. In any case, the table of differences still functions as a quantitive measure of eigenworm relevance.

7 Conclusion

For the first task of report we have introduced and analyzed three distinct approaches to time-series based classification, with both classical and neural network-based methods. Based on both performance and computational efficiency, the best approach we found is based on a data reduction process which extracts easy to parse information from time series data. The performance of this approach was then verified with cross-validation, and we are overall satisfied with the results obtained. An option for improving performance would be to resort to boosting: our data reduction approach seems especially apt for this, as not only it achieves the ebst performance, but is computationally lighter (data only needs to be reduced once) which is a great advantage for boosting which runs the algorithms multiple times.

For the second task of the report, we performed a Leave One Out analysis to give a measure of the relevance of each eigenworm, before and after data reduction. In this case, the original data seems to suggest eigenworms 5 and 6 are particularly relevant for classification. No similar trends emerge after going through data reduction.

The code for all of the plots and tables present in this report are available in the associated Colab file. The file also contains some figures and plots which weren't included in the report for brevity.

References

[1] Eviatar Yemini et al. "A database of Caenorhabditis elegans behavioral phenotypes". en. In: *Nat Methods* 10.9 (July 2013), pp. 877–879.

[2] Greg J. Stephens et al. "Dimensionality and Dynamics in the Behavior of C. elegans". In: *PLOS Computational Biology* 4.4 (Apr. 2008), pp. 1–10. DOI: 10.1371/journal.pcbi.1000028. URL: https://doi.org/10.1371/journal.pcbi.1000028.

[3] Bendong Zhao et al. "Convolutional neural networks for time series classification". In: *Journal of Systems Engineering and Electronics* 28.1, 162 (2017), p. 162. DOI: 10.21629/JSEE.2017.01.18. URL: https://www.jseepub.com/EN/abstract/article_5940.shtml.

[4] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. USA: Cambridge University Press, 2014. ISBN: 1107057132.